

Competitive Programming

Section 49 Editorial: Convex Hull Trick And Li Chao Tree

Competitive Programming Course

2026-07-10

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Dynamic Line Minimum	4
2.1	Restatement	4
2.2	Algorithm and proof	4
2.3	Complexity	4
2.4	Full C++ solution	4
2.5	Full Python solution	4
3	B. Quadratic CHT DP	5
3.1	Restatement	5
3.2	Algorithm and proof	5
3.3	Complexity	5
3.4	Full C++ solution	5
3.5	Full Python solution	5
4	C. Dynamic Line Maximum	6
4.1	Restatement	6
4.2	Algorithm and proof	6
4.3	Complexity	6
4.4	Full C++ solution	6
4.5	Full Python solution	6
5	Testing discussion	7

1 Editorial overview

1.1 Problem-specific proof map

A. Dynamic Line Minimum. A Li Chao node keeps the line best at its midpoint; the only line that can beat it on one half is recursed into that half, so every query path sees the minimum candidate.

B. Quadratic CHT DP. Expanding the square turns each earlier state into a line queried at $x[i]$. Thus the Li Chao answer is exactly the minimizing DP transition plus the query-only terms.

C. Dynamic Line Maximum. Negating all line values reduces maximum queries to the same minimum-line invariant used in A.

1.2 Detailed solution guide

A. Static Lines, Minimum Query. Each inserted line represents one candidate linear function $y = mx + b$. A query asks for the minimum value among all candidates at a given x . The local solution evaluates all lines so the definition is visible in the code. In larger constraints, the same set of lines is stored in a convex hull or Li Chao tree so the minimum is found faster.

B. DP to Lines. A common transition is $dp[i] = \min_j(dp[j] + a[j] * x[i] + b[j])$. For each previous j , create a line with slope $a[j]$ and intercept $dp[j] + b[j]$; querying at $x[i]$ gives exactly the best transition into i . The proof is just algebra: every possible previous state becomes one line, and no line represents an illegal state.

C. Li Chao Tree Query Model. A Li Chao tree stores, for each segment of the coordinate domain, a line that is currently best at that segment midpoint. Insertion compares the new line and old line at endpoints and midpoint, keeps the better midpoint line, and recurses only where the other line can still win. The local code may scan all lines, but the invariant explains why the optimized tree answers point queries in logarithmic time.

Common pitfalls. Equal slopes should keep only the better intercept. Use 64-bit arithmetic or wider if coordinates and slopes are large. Decide up-front whether the task asks for minimum or maximum; signs flip easily.

The common model is a set of lines.

- A and C maintain lines $y=mx+b$ and query the minimum or maximum at a point. The local limits allow a direct scan, which is the clearest correctness reference.
- B expands $(x_i - x_j)^2$ so each previous DP state j becomes a line with slope $-2*x_j$ and intercept $dp[j] + x_j^2$.

For larger external tasks, replace the direct scan by a Li Chao tree or a monotone convex hull. The mathematical reduction from candidate states to lines is unchanged.

2 A. Dynamic Line Minimum

2.1 Restatement

Add lines $y=mx+b$ and answer minimum value at query points.

2.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. \square

2.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

2.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct Line{long long m,b; long long get(long long x){return m*x+b;}};
```

2.5 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    q=data[0]; idx=1; lines=[]; out=[]
    for _ in range(q):
        t=data[idx]; idx+=1
        if t==1:
            m,b=data[idx],data[idx+1]; idx+=2; lines.append((m,b))
        else:
            x=data[idx]; idx+=1; out.append(str(min(m*x+b for m,b in lines)))
    print("\n".join(out))
if __name__=="__main__": main()
```

3 B. Quadratic CHT DP

3.1 Restatement

Compute $dp[i] = \min_j dp[j] + (x_i - x_j)^2 + c$ for increasing x .

3.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. \square

3.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

3.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n;long long
```

3.5 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,c=data[0],data[1]; x=data[2:]; dp=[0]*n
    for i in range(1,n):
        dp[i]=min(dp[j]+(x[i]-x[j])**2+c for j in range(i))
    print(dp[-1])
if __name__=="__main__": main()
```

4 C. Dynamic Line Maximum

4.1 Restatement

Add lines and answer maximum value at query points.

4.2 Algorithm and proof

The task-specific reduction in the proof map maps legal original choices to exactly the generated states, and the final operation reads the requested optimum or classification. \square

4.3 Complexity

The bound is the concrete one stated for this task in the proof map; the code performs only its enumerations, searches, or transitions.

4.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct Line{long long m,b; long long get(long long x){return m*x+b;}};
```

4.5 Full Python solution

```
import sys
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    q=data[0]; idx=1; lines=[]; out=[]
    for _ in range(q):
        t=data[idx]; idx+=1
        if t==1:
            m,b=data[idx],data[idx+1]; idx+=2; lines.append((m,b))
        else:
            x=data[idx]; idx+=1; out.append(str(max(m*x+b for m,b in lines)))
    print("\n".join(out))
if __name__=="__main__": main()
```

5 Testing discussion

Stress every line query against evaluating all inserted lines. Equal slopes need an explicit keep-better-intercept rule. For quadratic DP, expand $(x_i - x_j)^2$ before choosing slope and intercept; a sign error changes min to max behavior. Test repeated x , negative values, and a single line.

Random tests compare C++ and Python against fixed expected outputs generated from the reference implementation. Important cases include singletons, negative numbers where allowed, repeated values, and boundary query sizes.